How-To: MCP Servers

ragent speaks the **Model Context Protocol (MCP)** directly. You install an MCP server by whatever means it ships (npm, `uvx`/pip, or a released binary), point ragent at it in `ragent.json`, and its tools surface to the model as ordinary tools.

This document is the practical manual for installing MCP servers and wiring them into ragent: the transport model, how to install the well-known servers, how discovery works, the `mcp` config block, the enable/disable ledger, and how to call the tools.

For the `/mcp` slash command itself see `docs/howtos/slashcommands/mcp.md`; for the `mcp_tool` tool see `docs/howtos/tools/mcp.md`. The config field table lives in `docs/howtos/config.md` §7.7.

Table of Contents

- 1. Overview
- 2. Installing MCP Servers
- 3. Discovery
- 4. Configuring Servers
- 5. Enabling and Disabling
- 6. Calling MCP Tools
- 7. Plugin-Contributed Servers
- 8. Connector-Contributed Servers
- 9. Config Locations and Merge
- 10. Troubleshooting
- 11. Reference

---

1. Overview

MCP is ragent's transport for reaching external systems. Each server is defined by an id and one of three transports:

Transport	`type`	Requirement	Use case
stdio	`"stdio"`	`command` (+ optional `args`, `env`)	A local process launched and spoken to over stdin/stdout.
SSE	`"sse"`	`url` (+ optional `headers`)	A remote Server-Sent Events endpoint.
HTTP	`"http"`	`url` (+ optional `headers`)	A remote plain HTTP request/response endpoint.

A server's tools are registered into the session under the name `mcp_<server>_<tool>`, and are callable from chat through the `mcp_tool` wrapper. The permission system treats them as its own category (`mcp`), exactly like any other tool.

Three things can contribute a server to a session:

1. a `mcp.<id>` entry in `ragent.json` (what this doc is mostly about);
2. an enabled **plugin** whose manifest declares `mcpServers` (bridged as `<plugin-id>.<server>`);
3. an enabled **connector** whose descriptor names MCP servers (bridged as `<connector-id>.<server>`).

All three feed the same client and the same enable/disable ledger.

---

2. Installing MCP Servers

ragent does not install servers for you - install the server itself first, then add a `mcp` entry (or let `/mcp discover` find it). There are three common install routes.

Route A - Node / `npx` (no install step needed)

Most official servers are npm packages. You can run them straight through `npx` without a global install, and `npx` will fetch the package on first use:

```
npx -y @modelcontextprotocol/server-filesystem /home/user/docs
```

For a server you use often, install it globally so it is on `PATH` (this is also what discovery scans for):

```
npm install -g @modelcontextprotocol/server-filesystem
```

Route B - Python / `uvx` (or pip)

The reference Python servers ship an `mcp-server-<name>` executable entry point. `uvx` runs one without a persistent install:

```
uvx mcp-server-git
```

Or install it so it lands on `PATH`:

```
pip install mcp-server-git
```

Route C - Drop a definition into the server directory

For a server that is not one of the known executables, or that needs custom `args`/`env`, write a definition file under `~/.config/ragent/servers/` and let `/mcp discover` find it - no `ragent.json` edit. See section 3 for the two accepted file shapes.

```
mkdir -p ~/.config/ragent/servers/mytool
cat > ~/.config/ragent/servers/mytool/server.json <<'EOF'
{
  "name": "My Tool",
  "command": "npx",
  "args": ["-y", "@example/mcp-mytool"],
  "env": { "MYTOOL_TOKEN": "..." }
}
EOF
```

Known server executables

`/mcp discover` (section 3) looks for these exact executable names on `PATH`, in npm global directories, and in the MCP registry directories:

Server id	Executable(s) searched
`filesystem`	`mcp-server-filesystem`
`github`	`mcp-server-github`, `gh-mcp`
`git`	`mcp-server-git`
`postgres`	`mcp-server-postgres`
`sqlite`	`mcp-server-sqlite`
`memory`	`mcp-server-memory`
`brave-search`	`mcp-server-brave-search`
`fetch`	`mcp-server-fetch`
`puppeteer`	`mcp-server-puppeteer`
`slack`	`mcp-server-slack`
`google-drive`	`mcp-server-gdrive`
`google-maps`	`mcp-server-google-maps`
`sentry`	`mcp-server-sentry`
`sequential-thinking`	`mcp-server-sequential-thinking`
`everything`	`mcp-server-everything`
`time`	`mcp-server-time`
`aws-kb-retrieval`	`mcp-server-aws-kb-retrieval`
`exa`	`mcp-server-exa`

Any server not in this table can still be used - add it to `ragent.json` by hand (section 4). Discovery is a convenience, not a requirement.

---

3. Discovery

`/mcp discover` scans the machine for installed MCP servers and opens an interactive selection dialog. The scan checks, in order:

1. `PATH` - each known executable above.
2. **npm global** - the npm prefix's `node_modules`, including the `@modelcontextprotocol` scope.
3. **MCP registry directories** - `server.json` files, or direct `*.json` configs, under the ragent-native directory:
   - `~/.config/ragent/servers` - **the canonical ragent-native location** (see below).

   and the third-party locations, scanned for backwards compatibility:
   - `~/.claude/mcp-servers` - Claude Desktop
   - `~/.cline/mcp-servers` - Cline
   - `~/.mcp/servers` - legacy generic location (superseded by `~/.config/ragent/servers`)
   - `~/.config/mcp/servers`
   - `~/.config/claude/mcp-servers`

   The ragent-native directory is scanned first, so its definitions win de-duplication.

Results are de-duplicated by id, first find wins.

To see the scan reported per location - the label, the path (with a `(missing)` marker for an absent directory), and the number of definitions found at each, followed by the discovered servers and their launch commands - run **`/mcp config`**. It uses the same scan (`McpClient::discover_locations`) and is the quickest way to confirm which directory a definition is actually coming from.

The ragent-native server directory

Drop a server definition into `~/.config/ragent/servers/` (the canonical location; on macOS that is `~/Library/Application Support/ragent/servers/`, on Windows `%APPDATA%/ragent/servers/`) and `/mcp discover` picks it up on the next scan with no `ragent.json` edit. Two file shapes are accepted, both requiring a `command` field:

- a per-server subdirectory containing a `server.json` - the id is the directory name:

```
~/.config/ragent/servers/
+-- filesystem/
    +-- server.json
```

```
{
  "name": "Filesystem MCP Server",
  "command": "npx",
  "args": ["-y", "@modelcontextprotocol/server-filesystem", "/home/user/docs"],
  "env": {}
}
```

- a standalone `*.json` file - the id is the file stem:

```
~/.config/ragent/servers/git.json
```

```
{
  "name": "Git MCP Server",
  "command": "uvx",
  "args": ["mcp-server-git"]
}
```

`name` is optional and defaults to the id. A definition whose file has no `command` is skipped. The legacy `~/.mcp/servers/` directory still works and is scanned after the ragent-native location.

In the dialog, move the block cursor with `Up`/`Down` (or `k`/`j`; `Home`/`End` jump to the ends) and press `Space` to toggle the highlighted server's enabled state; `Esc` dismisses it and the list scrolls with a scrollbar when there are more servers than fit. Enabling a server **writes a new `mcp.<id>` entry into `.ragent/ragent.json`** (in the current project) with the discovered command, args, env, and `"disabled": false`, and records the choice in the global ledger; disabling it records the choice disabled and disconnects it. ragent reports:

`[ok] '<id>' enabled`

If a server id is already present in `ragent.json`, its own fields are kept (only a `disabled: true` override is lifted). Discovery never overwrites a hand-edited entry. A discovered server is written enabled, but the durable ledger (section 5) can still switch it off afterwards.

> Discovery is one way to get an entry into `ragent.json`. Editing the file by hand is equally supported and gives you full control over `args`, `env`, and `url`.

---

4. Configuring Servers

MCP servers live in the top-level `mcp` object of `ragent.json`, **keyed by server id** (the key is the id, there is no separate `name` field).

stdio server

```
{
  "mcp": {
    "filesystem": {
      "type": "stdio",
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-filesystem", "/home/user/docs"],
      "env": {}
    }
  }
}
```

`command` may be a bare executable name (resolved on `PATH`) or an absolute path. If it contains a `/` or `\` it must point at an existing file.

SSE / HTTP server

```
{
  "mcp": {
    "remote-api": {
      "type": "sse",
      "url": "https://mcp.example.com/sse",
      "headers": { "Authorization": "Bearer token123" },
      "disabled": false,
      "notification": "inject_summary"
    }
  }
}
```

`url` must start with `http://` or `https://`.

`McpServerConfig` fields

Field	Type	Default	Description
`type`	`stdio` / `sse` / `http`	`stdio`	Transport.
`command`	string	-	Executable (stdio). Required for stdio.
`args`	array of strings	`[]`	Arguments (stdio).
`env`	object	`{}`	Environment variables injected into the process (stdio).
`url`	string	-	Endpoint (sse/http). Required for sse/http.

Field	Type	Default	Description
`headers`	object	`{}`	HTTP headers on every request (sse/http).
`disabled`	bool	`false`	Configure the server but never start it.
`notification`	`none` / `inject_summary` / `inject_and_run`	`none`	How server-initiated push notifications are handled (see below).

Validation

Before a stdio server starts, its `command` and each of its `args` are checked: shell metacharacters (`|`, `;`, `&`, `$`, backtick, `(`, `)`, `{`, `}`, `<`, `>`) are rejected, and a path-like `command` must exist on disk. An sse/http server must have a valid `http(s)://` `url`. A server that fails validation is reported as `failed: <reason>` in the `/mcp` table rather than silently dropped.

Validation runs in **every** permission mode, including YOLO. YOLO relaxes the *permission* checks on what a tool may do; it does not relax the check that a server definition is well formed, because an invalid definition has no process to spawn and no endpoint to reach. Removing the YOLO short-circuit means a mistyped `command` is reported by name instead of failing later as an opaque `spawn: No such file or directory`.

Push notifications

`notification` controls what happens when a server pushes an event:

- `none` (default) - ignore it.
- `inject_summary` - inject a bounded summary into the chat without a model call.
- `inject_and_run` - inject a prompt and run one model turn in the full tool context.

Both non-`none` modes also require the top-level `piegap.mcp_notifications` gate to be enabled:

```
{ "piegap": { "mcp_notifications": true } }
```

Applying config changes

A server added to `ragent.json` while ragent is running is picked up on the next start. To rebuild the display list from the current config without restarting, run `/reload mcp`; to connect a newly added server live, run `/mcp connect <id>`.

---

5. Enabling and Disabling

Whether a server actually starts is **durable, user-owned state**, not merely a side effect of appearing in `ragent.json`. The choice is held in a global ledger:

`<global state dir>/mcp_state.json`

On Linux that is `~/.config/ragent/mcp_state.json`; if the platform state directory cannot be determined, ragent falls back to `.ragent/mcp_state.json` in the working directory. The ledger maps a server id to a boolean:

```
{ "servers": { "filesystem": false, "slack": true } }
```

The precedence rules:

- A server **absent** from the ledger is **enabled** - a newly added server starts with no extra step.
- `mcp.<id>.disabled: true` in `ragent.json` is a **hard override** that always disables the server, whatever the ledger says.
- Otherwise, the ledger decides.

Because the ledger is global, disabling a server once keeps it disabled in every project, and a plugin- or connector-contributed server (which has no `ragent.json` entry at all) can still be toggled off.

Toggling at runtime

Command	Effect
`/mcp` or `/mcp status`	Show the server table (id, enabled, status, transport, endpoint, tool count).
`/mcp connect <id>`	Enable in the ledger, connect live, register the server's tools.
`/mcp disconnect <id>`	Disable in the ledger, disconnect live.

A disconnected server's tools are refused even if a tool from an earlier connection is still registered; the refusal names the command that re-enables it. Enable/disable choices survive a restart.

---

6. Calling MCP Tools

Once a server is connected, its tools are registered as `mcp_<server>_<tool>` (with `-`, `.`, and `/` replaced by `_`) and the model calls them through the `mcp_tool` wrapper:

```
mcp_tool server="filesystem" tool="read_file" arguments={"path":"README.md"}
```

To see the live per-server inventory with the exact registry name each tool is callable under, run:

```
/plugins list --mcp
```

---

7. Plugin-Contributed Servers

An enabled plugin can declare MCP servers in its manifest. ragent bridges them into the session automatically - no `ragent.json` entry is needed. Bridged ids are prefixed with the plugin id to avoid collisions:

```
<plugin-id>.<server>
```

Both dialect spellings are accepted: `mcpServers` (Codex) and `mcp_servers` (Claude). A plugin can either inline the definitions:

```
{
  "mcpServers": {
    "mytool": { "command": "npx", "args": ["-y", "@example/mcp-mytool"] }
```

```
  }
}
```

or point at an external file whose own top-level `mcpServers` object supplies the entries (the Claude marketplace layout):

```
{ "mcpServers": "./mcp.json" }
```

A configured (`ragent.json`) server keeps its unprefixed id and wins any collision with a bridged one. Disabled plugins contribute nothing. See `docs/howtos/plugins.md` for the plugin manifest.

---

8. Connector-Contributed Servers

A **connector** is a catalogue-style integration that resolves to one or more MCP servers, bridged as `<connector-id>.<server>` through the same machinery. If your integration is available in a connector catalogue, prefer `/connectors add <id>` over hand-editing `ragent.json` - connectors also carry their authentication shape. See `docs/howtos/connectors.md`.

---

9. Config Locations and Merge

`ragent.json` is loaded from, in order (later overrides earlier):

1. `<global config dir>/ragent.json` - on Linux `~/.config/ragent/ragent.json`.
2. `.ragent/ragent.json` in the current project directory.

The `mcp` object merges **per server id, last-wins**: a project entry with the same id replaces the global one, and ids only present in one layer are unioned.

The `--config <path>` CLI flag (and the `RAGENT_CONFIG` / `RAGENT_CONFIG_CONTENT` environment variables) override this resolution for a single run.

A stdio entry containing secrets in `args` or `env` is best kept in the user-global file for a single user; keep per-project endpoints and non-secret arguments in the project file so they travel with the repository.

---

10. Troubleshooting

- **A server shows `failed: <reason>` in `/mcp`** - the reason is the validation or connection error. Common causes: a stdio entry with no `command`; a path-like `command` that does not exist; shell metacharacters in the command or an argument; an sse/http `url` not starting with `http(s)://`.
- **A server connects then never answers** - every connect attempt is bounded (default 30 s) and retried once on timeout. A launcher such as `npx`/`npm exec` that stalls resolving a package fails within the bound with an actionable error. Raise the bound for a slow network with `RAGENT_MCP_CONNECT_TIMEOUT_SECS=<seconds>`.
- **A server is configured but not started** - check, in order: the ledger (`/mcp status`), and `mcp.<id>.disabled` in `ragent.json`. The `disabled` flag wins over the ledger.
- **Tools do not appear** - confirm the server is connected (`/mcp`) and that its tools are registered (`/plugins list --mcp`). A server with no tools prints `tools: 0` and is still connected.

- **A config edit had no effect** - run `/reload mcp` to rebuild the display list from config, then `/mcp connect <id>` to connect a newly added server. A process that never started needs a restart.
- **A server in `~/.config/ragent/servers/` is not discovered** - the definition file must contain a top-level `command` (a file without one is skipped silently). Use `server.json` inside a per-server subdirectory, or a standalone `*.json` file; the id is the directory name or file stem respectively.
- `/doctor` - includes MCP connectivity in its diagnostics.

---

11. Reference

- `docs/howtos/slashcommands/mcp.md` - the `/mcp` command.
- `docs/howtos/tools/mcp.md` - the `mcp_tool` tool and the connection lifecycle.
- `docs/howtos/slashcommands/reload.md` - `/reload mcp`.
- `docs/howtos/config.md` §7.7 - the `mcp` config field table.
- `docs/howtos/plugins.md` - plugin `mcpServers` bridging.
- `docs/howtos/connectors.md` - connector-based integrations.
- `docs/howtos/permissions.md` - how MCP tools are permitted.